Cluster Architecture

Weight: 2

SECTION: ARCHITECTURE, INSTALL AND MAINTENANCE

For this question, please set the context to `cluster3` by running:

```
kubectl config use-context cluster3
```

Run a pod called `alpine-sleeper-cka15-arch` using the `alpine` image in the `default` namespace that will sleep for `7200` seconds.

- [ ] `alpine` pod created?

Task

SECTION: ARCHITECTURE, INSTALL AND MAINTENANCE

For this question, please set the context to `cluster1` by running:

```
kubectl config use-context cluster1
```

There is a script located at `/root/pod-cka26-arch.sh` on the `student-node`. Update this script to add a command to filter/display the label with value `component` of the pod called `kube-apiserver-cluster1-controlplane` (on `cluster1`) using `jsonpath`.

Task

SECTION: ARCHITECTURE, INSTALL AND MAINTENANCE

For this question, please set the context to `cluster1` by running:

```
kubectl config use-context cluster1
```

Create a service account called `deploy-cka20-arch`. Further create a cluster role called `deploy-role-cka20-arch` with permissions to `get` the `deployments` in `cluster1`.

Finally create a cluster role binding called `deploy-role-binding-cka20-arch` to bind `deploy-role-cka20-arch` cluster role with `deploy-cka20-arch` service account.

Task

SECTION: ARCHITECTURE, INSTALL AND MAINTENANCE

For this question, please set the context to `cluster2` by running:

```
kubectl config use-context cluster2
```

On the `student-node`, a Helm chart repository is given under the `/opt/` path. It contains the files that describe a set of Kubernetes resources that can be deployed as a single unit. The files have some issues. Fix those issues and deploy them with the following specifications: -

1. The release name should be `webapp-color-apd`.
2. All the resources should be deployed on the `frontend-apd` namespace.
3. The service type should be `node port`.
4. Scale the deployment to `3`.
5. Application version should be `1.20.0`.

`NOTE` : - Remember to make necessary changes in the `values.yaml` and `Chart.yaml` files according to the specifications, and, to fix the issues, inspect the template files.

Servicing and Networking

Weight: 6

SECTION: SERVICE NETWORKING

For this question, please set the context to `cluster1` by running:

```
kubectl config use-context cluster1
```

Create an **HPA** for a deployment named `frontend-deployment` in the **default** namespace. The **HPA** should scale the deployment based on **CPU** utilization, maintaining an average **CPU** usage of `70%` across all pods. Set the **minimum number of replicas** to `2` and the **maximum** to `10`.

Task

SECTION: SERVICE NETWORKING

For this question, please set the context to `cluster3` by running:

```
kubectl config use-context cluster2
```

Create an **HTTPRoute** named `web-route` in the **nginx-gateway namespace** that directs traffic from the `web-gateway` to a backend service named `web-service` on port `80` and ensures that the route is applied only to requests with the **hostname** `cluster2-controlplane`.
To test the configuration, SSH into the cluster2-controlplane node and run the following command:

```
curl http://cluster2-controlplane:30080
```

Task

SECTION: SERVICE NETWORKING

For this question, please set the context to `cluster1` by running:

```
kubectl config use-context cluster1
```

Create a nginx pod called `nginx-resolver-cka06-svcn` using image `nginx`, expose it internally with a service called `nginx-resolver-service-cka06-svcn`.

Test that you are able to look up the service and pod names from within the cluster. Use the image: `busybox:1.28` for dns lookup. Record results in `/root/CKA/nginx.svc.cka06.svcn` and `/root/CKA/nginx.pod.cka06.svcn`

For this question, please set the context to `cluster3` by running:

```
kubectl config use-context cluster3
```

There is a deployment `nginx-deployment-cka04-svcn` in cluster3 which is exposed using service `nginx-service-cka04-svcn`.

Create an **ingress resource** `nginx-ingress-cka04-svcn` to load balance the incoming traffic with the following specifications:

- `pathType: Prefix` and `path: /`
- Backend Service Name: `nginx-service-cka04-svcn`
- Backend Service Port: `80`
- `ssl-redirect` is set to `false`

Storage

For this question, please set the context to `cluster1` by running:

```
kubectl config use-context cluster1
```

A persistent volume called `papaya-pv-cka09-str` is already created with a storage capacity of `150Mi`. It's using the `papaya-stc-cka09-str` storage class with the path `/opt/papaya-stc-cka09-str`.

Also, a persistent volume claim named `papaya-pvc-cka09-str` has also been created on this cluster. This PVC has requested `50Mi` of storage from `papaya-pv-cka09-str` volume.

Resize the PVC to `80Mi` and make sure the PVC is in `Bound` state.

Task

SECTION: STORAGE

For this question, please set the context to `cluster1` by running:

```
kubectl config use-context cluster1
```

Create a storage class with the name `banana-sc-cka08-str` as per the properties given below:

- Provisioner should be `kubernetes.io/no-provisioner`,

- Volume binding mode should be `WaitForFirstConsumer`.

- Volume expansion should be enabled.

Workloads and Scheduling

Create a new deployment called `ocean-tv-wl09` in the default namespace using the image `kodekloud/webapp-color:v1`.
Use the following specs for the deployment:

1. Replica count should be `3`.

2. Set the **Max Unavailable to 40% and Max Surge to 55%.**

3. Create the deployment and ensure all the pods are ready.

4. After successful deployment, upgrade the deployment image to `kodekloud/webapp-color:v2` and inspect the deployment rollout status.

5. Check the rolling history of the deployment and on the `student-node`, save the `current` revision count number to the `/opt/revision-count.txt` file.

6. Finally, perform a rollback and revert back the deployment image to the older version.

In the `dev-wl07` namespace, one of the developers has performed a rolling update and upgraded the application to a newer version. But somehow, application pods are not being created.

To get back the working state, `rollback` the application to the previous version .

After rolling the deployment back, on the `controlplane` node, save the image currently in use to the `/root/rolling-back-record.txt` file and increase the replica count to the `5`.

You can SSH into the `cluster1` using `ssh cluster1-controlplane` command.

Troubleshooting

The `pink-depl-cka14-trb` Deployment was scaled to `2` replicas; however, the current replica count is still `1`.

Troubleshoot and fix this issue. Make sure the `CURRENT` count is equal to the `DESIRED` count.

You can SSH into the `cluster4` using `ssh cluster4-controlplane` command.

The `db-deployment-cka05-trb` deployment is having `0` out of `1` PODs ready.

Figure out the issues and fix the same but make sure that you do not remove any DB related environment variables from the deployment/pod.

It appears that the `black-cka25-trb` deployment in `cluster1` isn't up to date. While listing the deployments, we are currently seeing `0` under the `UP-TO-DATE` section for this deployment. Troubleshoot, fix and make sure that this deployment is up to date.

The `purple-app-cka27-trb` pod is an nginx based app on the container port `80`. This app is exposed within the cluster using a `ClusterIP` type service called `purple-svc-cka27-trb`.

There is another pod called `purple-curl-cka27-trb` which continuously monitors the status of the app running within `purple-app-cka27-trb` pod by accessing the `purple-svc-cka27-trb` service using `curl`.

Recently we started seeing some errors in the logs of the `purple-curl-cka27-trb` pod.

Dig into the logs to identify the issue and make sure it is resolved.

`Note:` You will not be able to access this app directly from the `student-node` but you can `exec` into the `purple-app-cka27-trb` pod to check.

The `yello-cka20-trb` pod is stuck in a `Pending` state. Fix this issue and get it to a `running` state. Recreate the pod if necessary.

Do not `remove` any of the existing `taints` that are set on the cluster nodes.